

auth-modular — Application Flow Report

Laravel 13 · PHP 8.3 · modular monolith · RabbitMQ (CloudAMQP) | Snapshot of branch `main` , 2026-09-20
(includes 3 uncommitted files)

1. The short version

This is a **Laravel modular monolith**: one deployable app, split into domain modules (`Auth` , `Users` , `Notifications` , `Audit`) plus a `Shared` module that holds the messaging plumbing. Modules do **not** call each other directly. They talk by publishing and consuming **events through RabbitMQ**.

Only one vertical slice exists so far: **“user registered”**. `Auth` publishes an `auth.user.registered` event, RabbitMQ routes it to a queue, and a worker in `Notifications` consumes it and logs it. There is no real registration endpoint yet, so the event is triggered from an Artisan test command. Everything else (`Users`, `Audit`, HTTP controllers, database persistence for modules) is empty scaffolding.

Important: the consumer command `rabbitmq:consume-notifications` cannot start right now, because `UserRegisteredHandler` is in a folder that doesn’t match its namespace (PHP can’t autoload it). I confirmed this by running it. Details and the fix are in section 8, issue A.

2. Status at a glance

Piece	Status	Notes
Laravel skeleton, module auto-discovery	WORKS	<code>ModuleServiceProvider</code> finds and registers every module provider.
HTTP routes	WORKS	Only <code>/</code> , <code>/up</code> (health) and <code>GET /api/v1/auth/test</code> .
Messaging contracts (<code>Message</code> , publisher / consumer / topology interfaces)	WORKS	Transport-agnostic; application code never touches <code>php-amqplib</code> .
RabbitMQ adapters (connection, publisher, topology)	WRITTEN	Not exercised by me: I did not run anything that talks to your broker.
Consumer retry path	FLAWED	Retried messages are dropped instead of returning to the queue (issue C).
<code>rabbitmq:consume-notifications</code> (Notifications worker)	BROKEN	Handler class not autoloadable (issue A).
Registration / login endpoints, database persistence per module	NOT STARTED	<code>Auth</code> has only the test route; <code>User</code> model + migrations are Laravel defaults.
<code>Users</code> and <code>Audit</code> modules	EMPTY	Provider + empty route files + empty layer folders.
Automated tests	NONE	Only the two stock Laravel example tests.

3. How the app boots and finds its modules

1. A request (or an Artisan command) enters through `public/index.php` or `artisan` and loads `bootstrap/app.php`.
2. `bootstrap/app.php` registers the base routes (`routes/web.php` , `/up`) and the console routes.
3. `bootstrap/providers.php` lists two providers: `AppServiceProvider` (empty) and `ModuleServiceProvider`.
4. `ModuleServiceProvider::register()` globs `app/Modules/*/Providers/*ServiceProvider.php`, builds the class names, sorts them and calls `$this->app->register()` on each. **Adding a module needs no wiring**: create the folder and a provider.
5. Each module provider's `boot()` calls `registerRoutes()`, which loads its `Routes/api.php` under the `api` middleware and the `/api` prefix (and `Routes/web.php` under `web`, except `Auth`, which is API-only).
6. `SharedServiceProvider::register()` binds the three messaging interfaces as singletons to their RabbitMQ implementations. This is the one place that decides “messaging = RabbitMQ”.

The modules

Auth partial

Owns the `UserRegistered` domain event and `AuthEventPublisher` (turns it into a message).
API route: `/api/v1/auth/test`.

Notifications partial

Consumes events. `NotificationEventConsumer`
→ `NotificationEventDispatcher` →
`UserRegisteredHandler` (currently just logs).

Shared done for now

Messaging ports (`Application/Messaging`) and RabbitMQ adapters
(`Infrastructure/Messaging/RabbitMQ`).

Users · Audit empty

Scaffolding only. No consumers or events yet.

Anatomy of a module

```
app/modules/<Module>/
├── Domain/      Contracts · Enums · Events · Exceptions    (pure business concepts)
├── Application/ Actions · DTOs · Services · Handlers        (use cases, orchestration)
├── Infrastructure/ Services · (repositories, adapters)      (talks to the outside
world)
├── Presentation/ Http/Controllers · Requests · Resources   (HTTP layer)
├── Providers/    <Module>ServiceProvider.php              (bindings + route loading)
└── Routes/      api.php · web.php
```

Dependencies point inward: Presentation and Infrastructure depend on Application and Domain, never the reverse. That is why `Auth` depends on the `MessagePublisherInterface` port, not on RabbitMQ.

4. The messaging layer (the heart of the app so far)

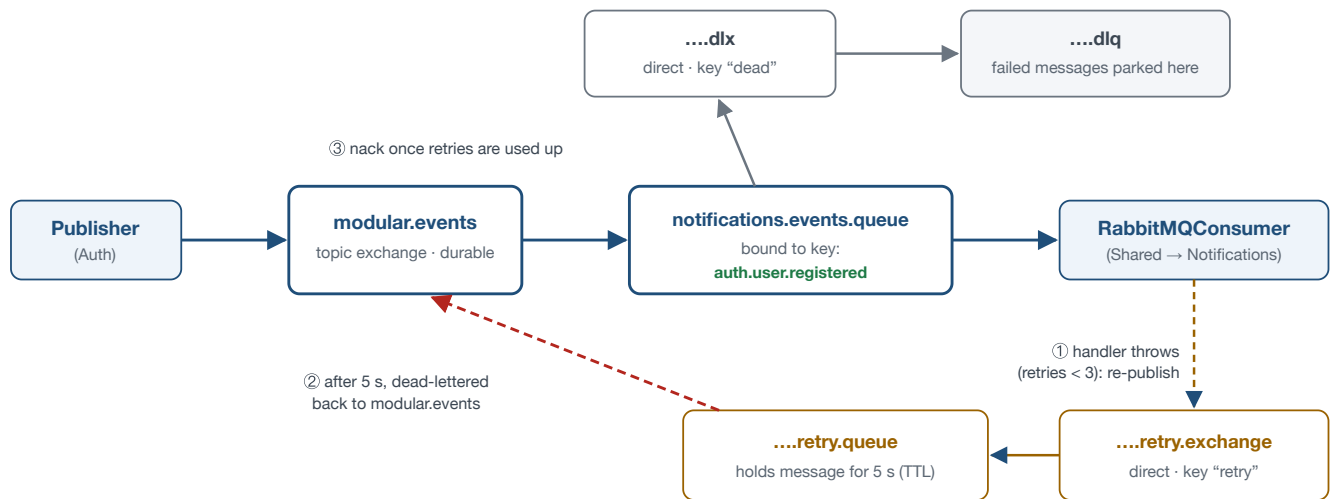
Ports (interfaces) and the message envelope: `Shared/Application/Messaging`

Type	Role
<code>Message</code>	Immutable envelope: <code>name</code> (also the routing key, e.g. <code>auth.user.registered</code>), <code>payload</code> , <code>headers</code> , <code>messageId</code> , <code>correlationId</code> .
<code>MessagePublisherInterface</code>	<code>publish(Message)</code> : used by modules that emit events.
<code>MessageConsumerInterface</code>	<code>consume(queue, handler)</code> : runs forever; handler returns normally $\Rightarrow$ ack, throws $\Rightarrow$ retry / dead-letter.
<code>MessageTopologyInterface</code> + <code>QueueTopology</code>	Declares exchanges, queues and bindings. A <code>QueueTopology</code> is “a queue name + the routing keys it listens to”.

Adapters: `Shared/Infrastructure/Messaging/RabbitMQ`

Class	What it does
<code>RabbitMQConnection</code>	Reads <code>config/rabbitmq.php</code> (host, port, user, password, vhost, SSL from <code>.env</code>). Uses a TLS connection when <code>RABBITMQ_SSL=true</code> (yours is on, port 5671).
<code>RabbitMQPublisher</code>	Opens a connection, declares the durable topic exchange <code>modular.events</code> , publishes the message as persistent JSON with the message name as routing key, then closes.
<code>RabbitMQTopology</code>	Creates the exchange, and per queue: main queue, dead-letter exchange + queue, retry exchange + queue, and the bindings.
<code>RabbitMQConsumer</code>	Long-running loop: decode JSON $\rightarrow$ rebuild <code>Message</code> $\rightarrow$ call the handler $\rightarrow$ ack. On exception it retries up to 3 times, then nacks to the dead-letter queue.

What `rabbitmq:setup` creates on the broker



⚠ **Step ②:** the message re-enters with routing key "retry". Nothing is bound to that key, so it is dropped (issue C).

Figure 1: RabbitMQ topology created by `rabbitmq:setup` (names shown for the `notifications.events.queue` family)

Retry policy as coded: 1 original attempt + 3 retries = 4 attempts. `retry_count` travels inside the message headers, so it survives the trip through the retry queue. After the 4th failure the consumer `nack s` without requeue and RabbitMQ's dead-letter arguments move the message to `.dlq`.

5. End-to-end flow: “user registered”

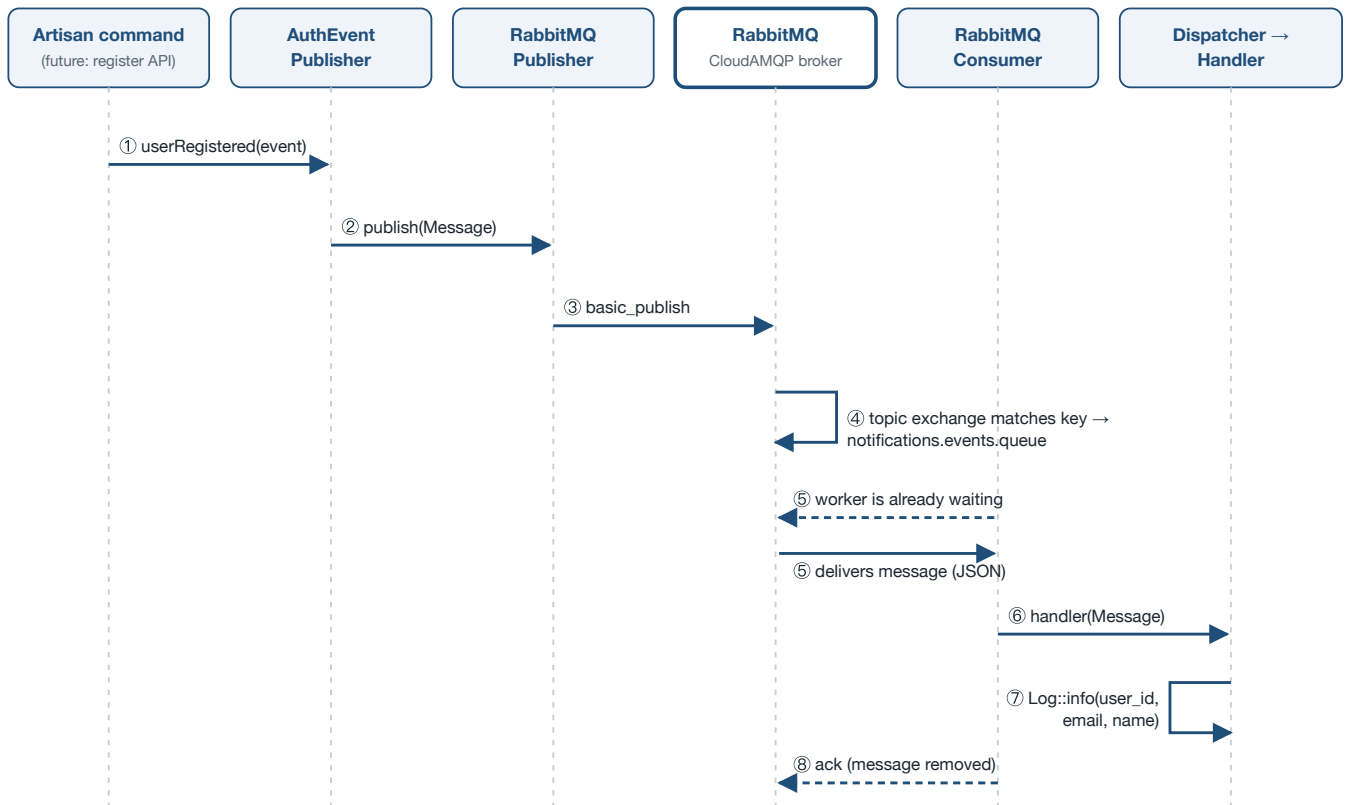


Figure 2: Sequence of the “user registered” event (steps ⑤ onward currently fail at ⑥, see issue A)

Step by step, with the code that does it

1. **Trigger.** `rabbitmq:publish-user-registered` (`PublishUserRegisteredTest`) builds a `UserRegistered` event with hard-coded values (`userId '123'`, `test@example.com`, `Ahmed`) and calls `AuthEventPublisher::userRegistered()`. In the real app, a registration action would do this after creating the user.
2. **Wrap in an envelope.** `AuthEventPublisher` creates a `Message` named `auth.user.registered` (the constant `UserRegistered::NAME`) whose payload is `{user_id, email, name}`. It only knows the `MessagePublisherInterface` port.
3. **Container resolves the port.** `SharedServiceProvider` bound the interface to `RabbitMQPublisher`, which serialises the message to JSON and publishes it to the `modular.events` exchange with routing key `auth.user.registered`. Note it opens and closes a TLS connection for each publish.
4. **Broker routes it.** The topic exchange delivers it to every queue whose binding matches. `rabbitmq:setup` bound `notifications.events.queue` to exactly this key, so that queue receives it. If the queue doesn't exist yet (setup not run), the message is silently discarded.
5. **Worker pulls it.** `rabbitmq:consume-notifications` resolves `NotificationEventConsumer`, which tells `RabbitMQConsumer` to consume `notifications.events.queue`. The consumer decodes the JSON back into a `Message`.
6. **Dispatch by name.** The consumer calls the closure passed by `NotificationEventConsumer`, which calls `NotificationEventDispatcher::dispatch()`. It looks the message name up in a map (`'auth.user.registered' -> UserRegisteredHandler`). Unknown names are ignored (and acked).

7. **Handle.** `UserRegisteredHandler` writes a `Log::info` entry to `storage/logs/laravel.log`. This is a placeholder: no email or notification is sent yet (`MAIL_MAILER=log`).
8. **Ack.** The handler returned normally, so the consumer acks and RabbitMQ deletes the message.

How to run the whole flow (once issue A is fixed)

```
# one-time / after topology changes
php artisan rabbitmq:test           # sanity check: can we reach the broker over TLS?
php artisan rabbitmq:setup         # create exchange, queue, DLQ, retry topology

# terminal 1: the worker (keeps running)
php artisan rabbitmq:consume-notifications

# terminal 2: fire the event
php artisan rabbitmq:publish-user-registered

# expected result
tail -f storage/logs/laravel.log    # "Handling user registered event" with
user_id/email/name
```

The project pins PHP 8.3 through `.envrc` (direnv). If `php` reports 8.2 or “not found”, run the commands from a shell where direnv has loaded.

6. Artisan command reference

Command	Class	Purpose
<code>rabbitmq:test</code>	<code>TestRabbitMQConnection</code>	Opens and closes a connection to the broker.
<code>rabbitmq:setup</code>	<code>SetupRabbitMQTopology</code>	Declares exchange/queues/bindings. Currently only <code>notifications.events.queue ← auth.user.registered</code> . This is where you add each new consumer’s queue.
<code>rabbitmq:publish-user-registered</code>	<code>PublishUserRegisteredTest</code>	Publishes a fake <code>UserRegistered</code> event through the Auth module.
<code>rabbitmq:consume-notifications</code>	<code>ConsumeNotificationEvents</code>	The real Notifications worker (long-running). broken, issue A
<code>rabbitmq:publish-test</code>	<code>PublishRabbitMQMessage</code>	Publishes a <code>test.message</code> . Nothing is bound to that key, so it goes nowhere (issue F).
<code>rabbitmq:consume-test</code>	<code>ConsumeRabbitMQTest</code>	Prints messages from <code>test.queue</code> .
<code>rabbitmq:consume-failure-test</code>	<code>ConsumeRabbitMQFailureTest</code>	Always throws, to exercise retry and dead-letter handling on <code>test.queue</code> .

7. Other configuration that shapes the flow

- **Database:** MySQL. Sessions, cache and Laravel's own queue driver are all set to `database`. The default `users`, `cache`, `jobs` migrations exist. RabbitMQ is *not* Laravel's queue driver here; it is used directly through the messaging layer.
- **RabbitMQ:** hosted on CloudAMQP, TLS on (port 5671), configured via `RABBITMQ_*` keys in `.env` and read by `config/rabbitmq.php`.
- **Errors:** `bootstrap/app.php` forces JSON error responses for `api/*` and JSON-expecting requests.
- **Git state:** the last commit covers the publisher and topology. The consumer command, `PublishUserRegisteredTest` and `NotificationEventConsumer` are still uncommitted.

8. Issues found while tracing the flow

A and B I reproduced by running code. C, D, E, F and G come from reading the code; I did not exercise the broker.

HIGH A. The Notifications handlers can't be autoloaded, so the worker won't start (reproduced)

`UserRegisteredHandler` and `NotificationEventHandler` declare namespace `App\Modules\Notifications\Application\Handlers`, but the files sit in `Notifications/Handlers/` (no `Application/`). PSR-4 expects the path to mirror the namespace. Running `class_exists()` returns `false`, and resolving `NotificationEventConsumer` throws `BindingResolutionException: Target class [...UserRegisteredHandler] does not exist`.

Fix: move both files to `Notifications/Application/Handlers/` (matches the layered layout of the other modules), then `composer dump-autoload`.

HIGH B. Folder is `app/modules`, code expects `app/Modules` (reproduced: works on macOS only)

Git tracks `app/modules` (lowercase), but every namespace is `App\Modules\...` and `ModuleServiceProvider` globs `app_path('Modules')`. macOS's case-insensitive filesystem hides this. On Linux (CI, Docker, production) no module would be discovered and no class would autoload.

Fix: `git mv app/modules app/Modules_tmp && git mv app/Modules_tmp app/Modules` (two steps, because macOS treats a one-step case rename as a no-op), then commit.

HIGH C. Retried messages are dropped instead of coming back (from reading; not tested on the broker)

On failure the consumer re-publishes the message to `...retry.exchange` with routing key `retry`. The retry queue dead-letters to `modular.events` after 5 s but sets no `x-dead-letter-routing-key`, so RabbitMQ reuses the message's key, which is `retry`. No queue is bound to `retry` on the topic exchange, so the message vanishes. In practice a transient failure loses the event, and the DLQ is only reached if a message is *already* at its retry limit.

Fix: make the retry queue dead-letter to the default exchange (`x-dead-letter-exchange: ''`) with `x-dead-letter-routing-key = the main queue name`, so the message goes straight back to `notifications.events.queue`. RabbitMQ refuses to change arguments on an existing queue, so delete the old `...retry.queue` before re-running `rabbitmq:setup`.

MEDIUM D. A malformed message can crash the worker in a loop

In `RabbitMQConsumer` the `json_decode(..., JSON_THROW_ON_ERROR)` and the `Message` construction sit outside the `try`. A non-JSON body (or one without `name`) throws out of the callback, kills the worker, and the unacked message is redelivered to the next worker start, repeating forever. There is also no `basic_qos` prefetch limit. **Fix:** move decoding inside the `try` and nack bad payloads straight to the DLQ.

MEDIUM E. The business side isn't built yet

No controller, action or DTO exists in any module. `UserRegistered` is only emitted by the test command; the `User` model and migration are Laravel's defaults (integer id, while the event carries `userId` as a string). `Users` and `Audit` are empty and have no queue or handler, so nothing yet reacts to events beyond the one log line.

LOW F. Test commands and the real topology don't line up

`rabbitmq:setup` only creates `notifications.events.queue`. The `test.queue` used by `consume-test` and `consume-failure-test` is never declared, and `rabbitmq:publish-test` sends `test.message`, which nothing is bound to. Those commands only work if you create and bind `test.queue` by hand.

LOW G. Small things

- `RabbitMQPublisher` opens a new TLS connection and re-declares the exchange on every publish. Fine for now; reuse a connection when publish volume grows.
- `RabbitMQConsumer` imports `Contracts\MessageConsumer`, a class that doesn't exist. Harmless (unused import), but worth deleting.
- The dispatcher hard-codes `'auth.user.registered'` instead of using `UserRegistered::NAME`. This avoids a compile-time dependency from Notifications to Auth, but it means the event name is duplicated in three places (Auth event, dispatcher, `rabbitmq:setup`).
- No tests cover any of this; the messaging layer is easy to fake through `MessagePublisherInterface`.

9. Suggested next steps

1. Fix A and B (both quick), then run the flow from section 5 to see the first end-to-end event.
2. Fix C (and D) before relying on retries or the DLQ.
3. Build the real registration action in `Auth` (request → DTO → action → create user → publish `UserRegistered`), replacing the test command.
4. Make `UserRegisteredHandler` actually notify (mail via the `log` mailer first), and give `Audit` its own queue bound to the same event to prove the fan-out works.
5. Add tests against a fake `MessagePublisherInterface`, and commit the three untracked files.